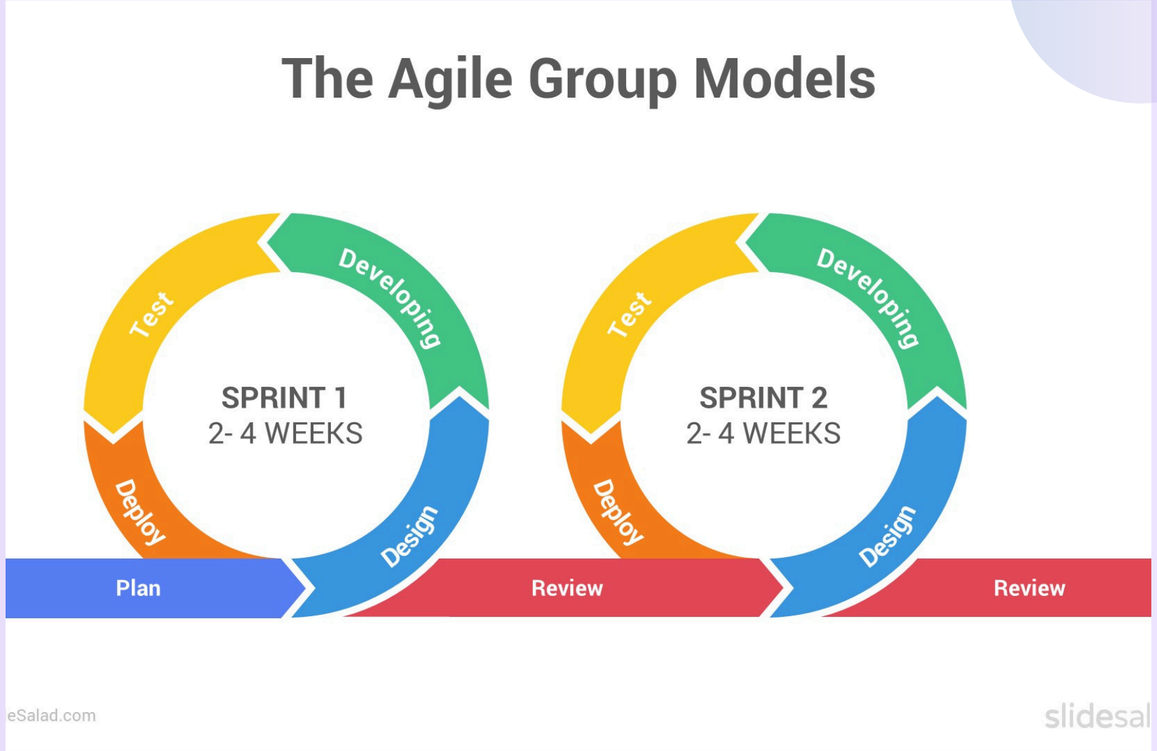
Agile Software Projects

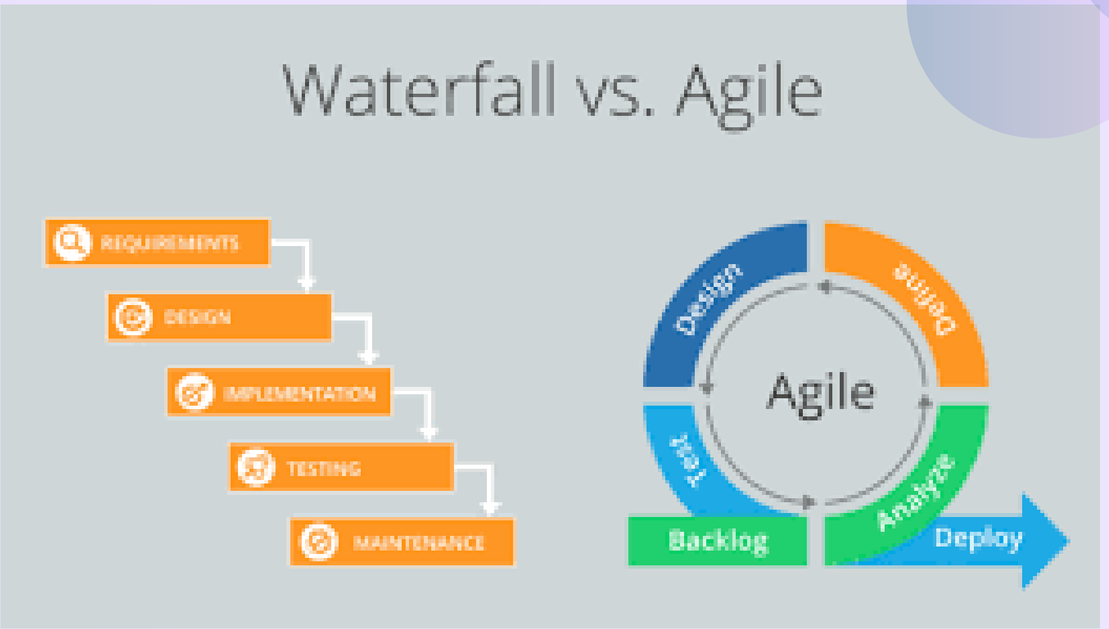
What is Agile?

- Agile is a software development approach focused on flexibility, collaboration, customer feedback, and rapid delivery.
- Unlike traditional methods, Agile delivers small, working parts of the software frequently, allowing teams to adapt quickly to changes.
- It values individuals and interactions, working software, customer collaboration, and responding to change.



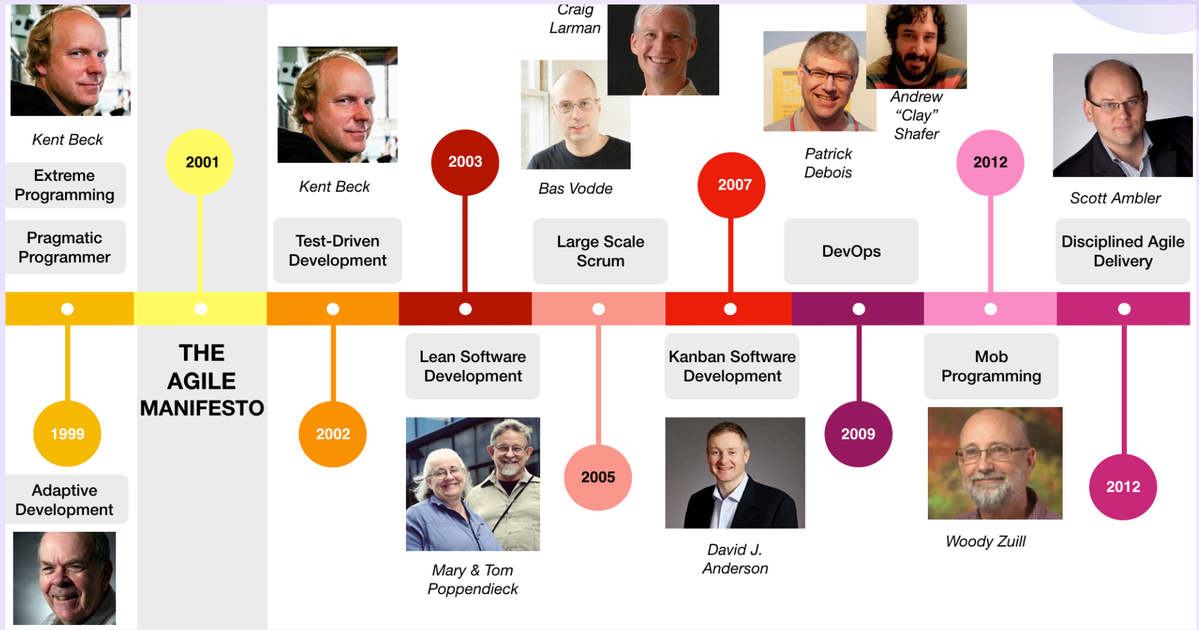
Why Agile Came Into Play?

- Traditional methods like Waterfall were rigid and failed when:
  - Requirements changed mid-project.
  - Customers weren't involved until the end.
  - Projects ran late or over budget.
  - Businesses needed a more adaptive and customer-focused approach.



History Of Agile

- 1990s: Frustration with heavyweight development models grew.
- 2001: 17 software developers met in Utah and created the Agile Manifesto.
- The manifesto had 4 core values and 12 principles.
- This marked the official birth of Agile.



Conclusion

Content:

- Agile transformed software development.
- It's now widely adopted in industries beyond tech (like marketing, education, even construction).
- It enables faster, better-aligned, and more successful project outcomes.

Agile Manifesto

The Agile Manifesto is the foundational document of Agile methodology

In 2001, 17 software developers met in Utah and created the Agile Manifesto.

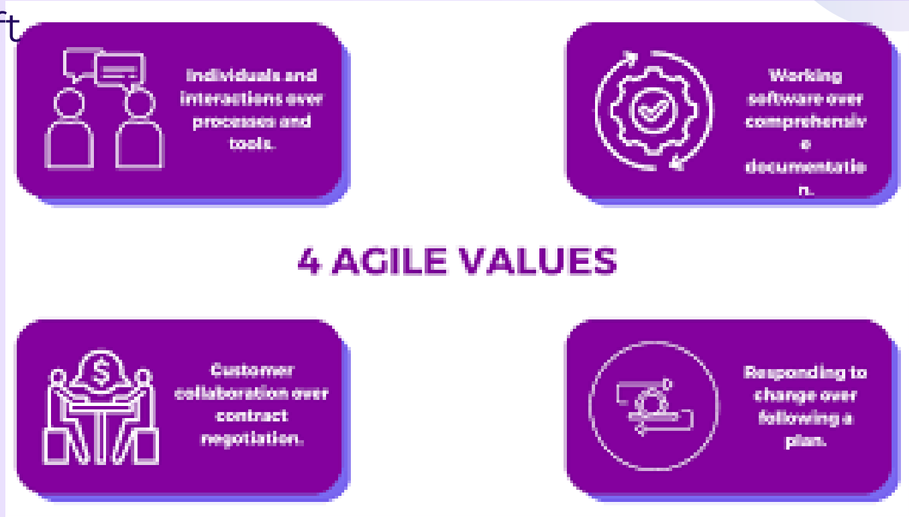
These pioneers came from different backgrounds (Extreme Programming, Scrum, DSDM, Crystal, etc.), but shared one belief:

“We need a better way to build software — one that delivers value early and adapts quickly to change.”

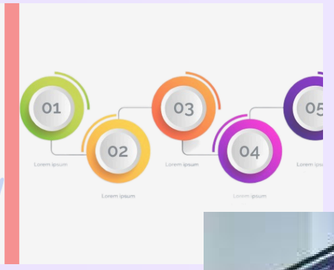
The Agile Manifesto didn't invent Agile practices but unified them under a common philosophy.

- Agile emphasizes people, collaboration, flexibility, and working products. The Manifesto presents four key values where the left side is prioritized more than the right:

We value...	VS Over...
Individuals and interactions	Processes and tools
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a fixed plan



- Individuals and Interactions over Processes and Tools
  - Agile believes great tools and strict processes are not enough. It's the team's communication, skills, and collaboration that truly drive success.



- Working Software over Comprehensive Documentation
  - While some documentation is necessary, Agile prefers to deliver a usable product quickly rather than wasting time on lengthy paperwork.

- Customer Collaboration over Contract Negotiation
  - Instead of locking clients into rigid contracts, Agile teams work closely and continuously with customers to build what they truly need.

- Responding to Change over Following a Plan
  - Plans are useful, but Agile welcomes change at any stage, allowing the product to evolve with user feedback.

The 12 Principles

These principles guide Agile teams in how to work effectively

1. Satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently (weeks rather than months).
4. Business people and developers must work together daily.
5. Build projects around motivated individuals.
6. Face-to-face conversation is the best form of communication.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective.

The 12 Principles

Agile Development Life Cycle - ADLC

The Agile Development Life Cycle (ADLC) is the step-by-step process used in Agile projects to plan, build, test, and deliver software incrementally and iteratively.

Unlike traditional methods where the entire product is delivered at the end, Agile delivers working software in smaller pieces (called iterations or sprints), allowing for continuous improvement, customer feedback, and flexibility to change.

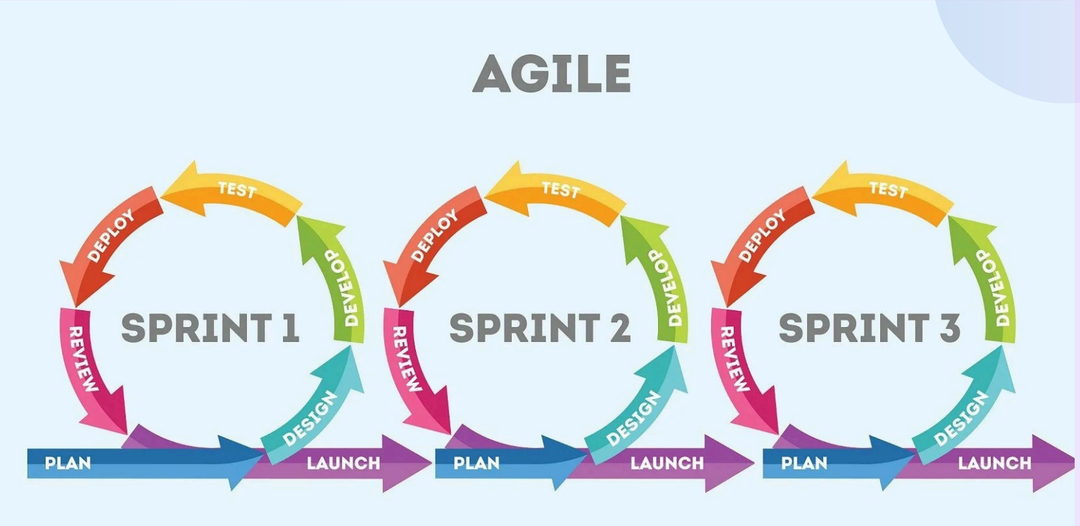
Agile doesn’t follow a rigid step-by-step pattern. Instead, the stages can overlap and repeat, depending on feedback and evolving requirements.

Overview

Agile is not a linear process. It’s iterative — meaning each sprint produces a potentially shippable product, and feedback from one iteration feeds the next.

Each sprint (usually 1–4 weeks) includes planning, development, testing, review, and release.

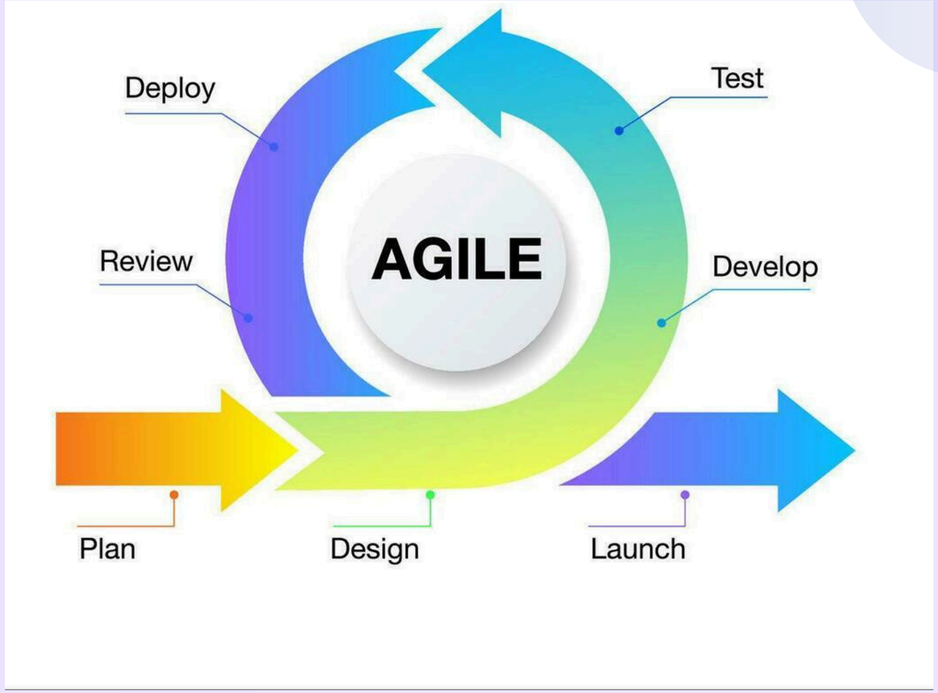
The Agile team works closely with customers and stakeholders throughout the process, making it collaborative, transparent, and adaptable.



ADLC Stages

key stages involved in ADLC, typically repeated every sprint:

1. Concept / Requirements Gathering & Sprint Planning
2. Design
3. Development / Coding
4. Testing
5. Release
6. Review & Retrospective

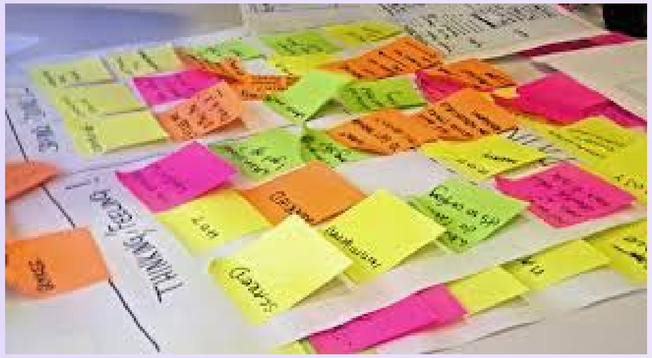


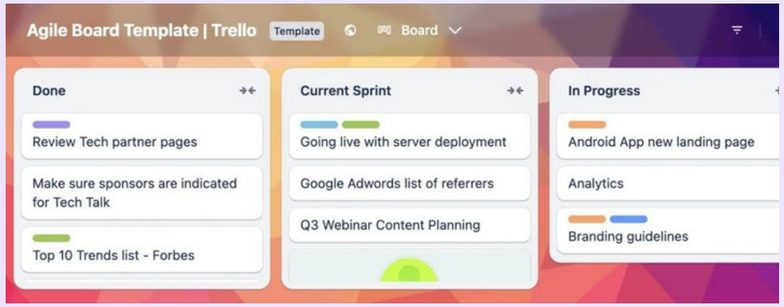
1. Concept / Requirements Gathering & Sprint Planning

- What happens?
  - Product owner collaborates with stakeholders to identify high-level business needs and prioritize features. Then the team selects items from the backlog to work on during the sprint and estimates how long tasks will take.

- Key outputs:
  - Product Backlog & Sprint backlog
  - User stories
  - Initial release plan
  - Definition of Done

- Central Activities:
  - Define “What” needs to be built
  - High-level planning
  - Break user stories into tasks
  - Assign work
  - Set sprint goals





2 Design

- What happens?
  - Basic technical design and UX/UI mockups are created based on sprint tasks. Lightweight and just enough to start building.

- Key outputs:
  - Simple design models
  - Architecture decisions

- Central Activities:
  - UI/UX wireframes
  - Data models
  - Architecture diagrams

3 Development / Coding

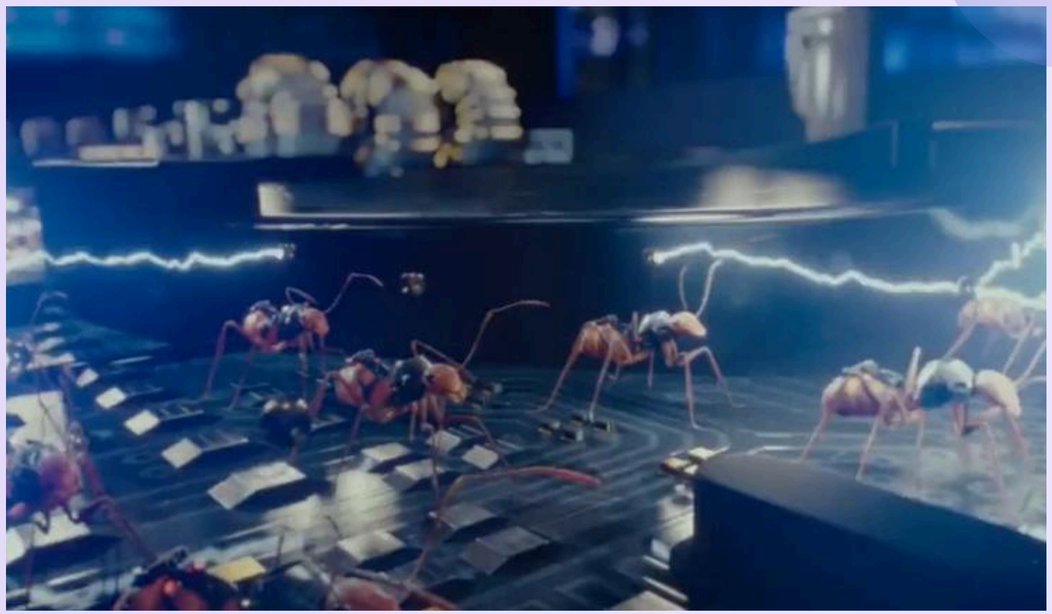
- What happens?
  - Developers build the features planned for the sprint, often working in pairs or small sub-teams.

- Key outputs:
  - Working code
  - Unit tests

- Central Activities:
  - Coding
  - Version control
  - Continuous integration

4 Testing

- What happens?
  - QA and developers test the functionality using both manual and automated techniques.
- Key outputs:
  - Test cases
  - Bug reports
  - Verified code
- Central Activities:
  - Unit testing
  - Integration testing
  - Regression testing
  - User Acceptance Testing (UAT)



5 Release

- What happens?
  - The product increment is delivered to users or stakeholders — can be deployed or just demoed.

- Key outputs:
  - Working product
  - Release notes

- Central Activities:
  - Deployment
  - Packaging
  - Documentation

6 Review & Retrospective

- What happens?
  - The team and stakeholders review the sprint outcome and reflect on how to improve in the next sprint.

- Key outputs:
  - Sprint Review: What was delivered
  - Sprint Retrospective: What to improve

- Central Activities:
  - Demo to stakeholders
  - Team discussion on what went well, what didn’t
  - Process improvements

Summary

Stage	*Main Activity*	*Key Outputs*
Concept & Sprint Planning	*Requirements, user storiesTask breakdown, estimation*	*Product Backlog Sprint Backlog*
Design	*Wireframes, architecture*	*Design blueprints*
Development	*Code creation*	*Working software*
Testing	*QA, bug fixing*	*Tested software*
Release	*Delivery of product increment*	*Deployed/ready feature*
Review & Retrospective	*Demo + reflection*	*Lessons & improvements*

Pros and cons

Agile is one of the most widely used software development methodologies today — known for its flexibility, speed, and customer-focused approach. However, like any method, Agile has both strengths and weaknesses.

Pros and Cons Of Agile

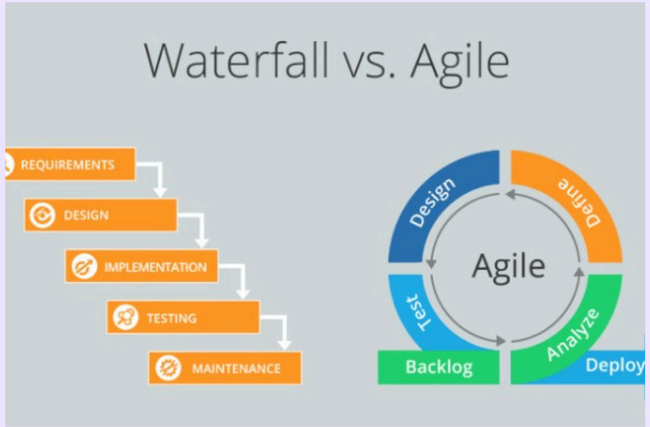
2025/06/15

PROS AND CONS	
PROS 👍	*CONS* 👎
Customer Involvement	*Requires Frequent Customer Input*
Flexibility & Adaptability	*Not Ideal for Fixed-Scope Projects*
Early and Continuous Delivery	*Documentation Can Be Minimal*
Improved Quality	*Can Be Hard to Scale*
Transparency	*Time-Consuming Meetings*
Better Risk Management	*Lack of Predictability for Final Product*
Team Collaboration & Ownership	*High Team Dependency*
Predictable Costs & Schedule	

Agile vs Traditional Software Development Approaches

Agile and traditional (also called plan-driven) development models follow very different paths to create software. Understanding their differences helps teams choose the right method based on project size, clarity, risk, and customer involvement.

Feature	***Agile Approach***	***Traditional Approach (e.g., Waterfall)***
Development Style	Iterative & Incremental	Linear & Sequential
Flexibility to Change	Very High	Low
Requirement Stability	Not required to be fully clear at the start	Must be clearly defined upfront
Risk Management	Continuous assessment & feedback	Risk assessed early but hard to handle new risks
User Involvement	Continuous collaboration & feedback	Limited to initial requirements
Delivery Speed	Early & frequent delivery of working software	Delivered at the end of full cycle
Documentation	Lightweight and just enough	Heavy and detailed documentation upfront
Testing	Continuous (with every iteration)	Only after development is complete
Team Structure	Cross-functional, self-organizing	Defined roles with hierarchical control
Client Satisfaction	High (customer-driven)	Varies (depends on initial accuracy)



Agile vs Other Models

Checklist					
Model	*Flexibility*	*Risk Management*	*User Involvement*	*Duration*	*Best Fit*
Agile (Scrum/XP)	Extremely High	Continuous feedback & iteration	Very High	Short Cycles	Uncertain, changing needs, fast delivery
Waterfall	Very Low	Early risk detection but costly to fix	Low	Long	Regulated, stable requirements
RAD	High	Short cycles + user feedback	Exceptionally High	Very Short	Fast UI-focused systems
Prototype	Very High	Great for early UI risk reduction	Very High	Varies	Unclear requirements, new ideas
Spiral	Very High	Excellent	Moderate to high	Long	Complex, high-risk projects
Iterative	High	Excellent continuous feedback	High	Short Cycles	Complex, evolving systems

When to Use Agile?

Agile is best suited for projects that:

- Have changing or unclear requirements
- Need early delivery and frequent updates
- Involve continuous client feedback
- Are user-focused (UI/UX heavy apps)
- Require team collaboration and adaptability
- Are not tightly regulated

When Agile May Not Be Suitable

Avoid Agile if:

The project has very strict regulations (e.g., government or defense)

Requirements are fully known and unlikely to change

You have a dispersed team with no access to regular collaboration

The client demands detailed documentation upfront

THE END

Members

Team Members

H.K.Y SANDEEPA -242093
R.A.S.THIVANKA - 242106
K.S. WICKRAMARATHNE - 242116
D.A.I.A. SENARATHNA - 242095
M.L.M. MAHALAXA - 242060
D.A.G.PATHIRANA - 242074
D..A.G.PATHIRANA - 242075
C.K.D.S.MANUKULASURIYA-242065
S.V. KAHINGALA - 242054
K.V.D.P.I.WIJEGUNARATHNA - 242118
M.R. AMARASEKARA - 242008

Content By

H.K.Y SANDEEPA -242093
R.A.S.THIVANKA - 242106
K.S. WICKRAMARATHNE - 242116
D.A.I.A. SENARATHNA - 242095
M.L.M. MAHALAXA - 242060
D.A.G.PATHIRANA - 242074
D..A.G.PATHIRANA - 242075
C.K.D.S.MANUKULASURIYA-242065
S.V. KAHINGALA - 242054
K.V.D.P.I.WIJEGUNARATHNA - 242118
M.R. AMARASEKARA - 242008

Graphics and slides by
H.K.Y. SANDEEPA - 242093

Thank You.